Apunte teórico - Lab. Algoritmos y Estructura de Datos (parte 2)

ÍNDICE:

1. ¿Qué es un vector?
2. Vectores estáticos
3. Recorrer un vector
4. Buscar elementos en un vector
5. Ordenar un vector
6. ¿Qué es una matriz?
7. Diagonales (principal y secundaria)
8. ¿Qué es una estructura (struct)?
9. Memoria dinámica
10. Vectores dinámicos

---

1. ¿QUÉ ES UN VECTOR?

Definición

Un vector (también llamado arreglo o array) es una estructura que permite almacenar varios datos del mismo tipo utilizando un único nombre.

Todos los elementos que guarda un vector deben ser del **mismo tipo de dato.**

Por ejemplo:

- Un vector de enteros guarda solamente enteros.
- Un vector de float guarda solamente float.
- Un vector de char guarda solamente char.

En palabras simples: Imaginemos una caja de un tipo de dato con “x” cantidad de separaciones.

Hasta ahora vimos variables:

int edad = 16;

Una variable es como una cajita que guarda un único valor.

Podemos pensarlo así:

edad

16

Pero...

¿Qué pasa si queremos guardar las edades de 30 alumnos?

¿Vamos a crear 30 variables?

int edad1;

int edad2;

int edad3;

...

int edad30;

Sería incómodo y poco práctico.

Para resolver este problema existen los vectores.

Un vector es como una fila de cajitas o una cajita grande con separadores.

edades

16	15	17	16	18

Todas las edades están guardadas bajo un mismo nombre:

edades

Declaración de un vector

Para poder crear o declarar un vector en C la sintaxis es:

tipo nombre[tamaño];

Ejemplo:

int edades[5];

Esto significa:

- El vector se llama edades.
- Puede almacenar 5 números enteros.
- Todavía no tiene valores asignados.

Podemos imaginarlo así:

edades

?	?	?	?	?

Asignar valores

Para poder asignar valores a cada **POSICIÓN** del vector hacemos:

edades[0] = 16;

edades[1] = 15;

edades[2] = 17;

edades[3] = 16;

edades[4] = 18;

Ahora el vector contiene:

Posicion	0	1	2	3	4
edades	16	15	17	16	18

Acceder a un elemento

Para acceder a un elemento utilizamos su posición.

Ejemplo:

printf("%d", edades[2]);

Resultado:

17

Porque en la posición 2 está almacenado el valor 17.

IMPORTANTE: Los vectores empiezan en 0

Este es uno de los errores más comunes. Muchos piensan que el primer elemento está en la posición 1. Pero en C (y la mayoría de lenguajes) no es así. Siempre comienza en: 0

Ejemplo completo

```
#include <stdio.h>
int main()
{
  int edades[5];
  edades[0] = 16;
  edades[1] = 15;
  edades[2] = 17;
  edades[3] = 16;
  edades[4] = 18;
  printf("%d\n", edades[0]);
  printf("%d\n", edades[2]);
  return 0;
}
```

Salida:

16

17

---

2. VECTORES ESTÁTICOS

Definición

Un vector estático es un vector cuyo tamaño se define al momento de escribir el programa y no puede modificarse durante la ejecución.

Cuando creamos un vector estático, debemos indicar cuántos elementos podrá almacenar.

Sintaxis

```
tipo nombre[tamaño];
```

Ejemplos:

```
int edades[30];
float notas[10];
char letras[5];
```

¿Qué significa el tamaño?

Si hacemos:

```
int edades[5];
```

Estamos creando un vector que puede almacenar exactamente 5 números enteros.

Podemos imaginarlo así:

edades

?	?	?	?	?

Hay lugar para 5 valores. Ni más. Ni menos.

¿Qué pasa si intento guardar más elementos?

Supongamos:

int edades[5];

Las posiciones válidas son:

0

1

2

3

4

Si intentamos hacer:

edades[5] = 20;

Estamos intentando acceder a una posición que no existe.

Esto produce un error lógico y puede generar comportamientos inesperados.

Por eso es muy importante respetar siempre el tamaño del vector.

Ventajas de los vectores estáticos

- Son simples de usar.
- Son rápidos.
- No requieren memoria dinámica.
- Son ideales cuando sabemos de antemano cuántos datos vamos a almacenar.

Ejemplo:

Si sabemos que un curso tiene exactamente 30 alumnos:

int notas[30];

es una buena solución.

Desventajas de los vectores estáticos

- El tamaño queda fijo.
- Si después necesitamos más espacio, no podemos agrandarlo.

Ejemplo:

int notas[30];

Si aparecen 5 alumnos más:

35 alumnos

El vector ya no alcanza.

Tendríamos que modificar el programa y volver a compilarlo.

Inicialización directa

También podemos crear el vector y cargarlo al mismo tiempo.

Ejemplo:

int numeros[5] = {10, 20, 30, 40, 50};

Visualmente:

Posicion	0	1	2	3	4
Valor	10	20	30	40	50

Ejemplo completo

```
#include <stdio.h>
int main()
{
   int numeros[5] = {10, 20, 30, 40, 50};
   printf("%d\n", numeros[0]);
   printf("%d\n", numeros[4]);
   return 0;
}
```

Salida:

10

50

Relación con las variables

Antes teníamos:

```
int nota1 = 8;
int nota2 = 10;
int nota3 = 7;
int nota4 = 9;
int nota5 = 6;
```

Ahora podemos hacer:

```
int notas[5];
```

Y guardar todas las notas dentro del mismo vector.

Por eso los vectores son tan importantes.

Permiten manejar grandes cantidades de información de forma organizada.

---

3. RECORRER UN VECTOR

Definición

Recorrer un vector significa visitar cada una de sus posiciones para poder leer, mostrar, modificar o procesar los valores almacenados.

¿Por qué necesitamos recorrer un vector?

Supongamos que tenemos:

```
int notas[5] = {8, 10, 7, 9, 6};
```

Si quisiéramos mostrar todas las notas podríamos hacer:

```
printf("%d\n", notas[0]);
printf("%d\n", notas[1]);
printf("%d\n", notas[2]);
```

```
printf("%d\n", notas[3]);
printf("%d\n", notas[4]);
```

Funciona. Pero si el vector fuera de 100 elementos ¿Vas a escribir 100 printf? No.

Por eso utilizamos ciclos.

La forma más común: usando for

```
for(int i = 0; i < 5; i++)
{
   printf("%d\n", notas[i]);
}
```

Salida:

8

10

7

9

6

Si hacemos:

notas[i]

queremos acceder a:

notas[0]

notas[1]

notas[2]

notas[3]

notas[4]

Simulación paso a paso

Supongamos:

```
int notas[5] = {8, 10, 7, 9, 6};
```

Y:

```
for(int i = 0; i < 5; i++)
{
   printf("%d\n", notas[i]);
}
```

Primera iteración

i = 0

Entonces:

```
printf("%d\n", notas[0]);
```

Muestra:

8

Luego:

i++

Ahora:

i = 1

Segunda iteración

printf("%d\n", notas[1]);

Muestra:

10

Tercera iteración

printf("%d\n", notas[2]);

Muestra:

7

Y así sucesivamente hasta llegar a:

i = 4

Patrón general

Cuando recorremos un vector casi siempre veremos algo parecido a esto:

```
for(int i = 0; i < tamaño; i++)
{
  // trabajar con vector[i]
}
```

Ejemplo:

```
for(int i = 0; i < 10; i+  +)
{
  printf("%d\n", numeros[i]);
}
```

Cargar un vector usando scanf:

Recorrer no solo sirve para mostrar. También sirve para cargar datos.

```
int edades[5];
for(int i = 0; i < 5; i++)
{
  printf("Ingrese una edad: ");
  scanf("%d", &edades[i]);
}
```

¿Qué está pasando?

Primera vuelta:

scanf("%d", &edades[0]);

Segunda vuelta:

scanf("%d", &edades[1]);

Tercera vuelta:

scanf("%d", &edades[2]);

Y así sucesivamente.

Modificar todos los elementos

Supongamos:

```
int numeros[5] = {1, 2, 3, 4, 5};
```
Queremos multiplicarlos por 2.
```
for(int i = 0; i < 5; i++)
{
   numeros[i] = numeros[i] * 2;
}
```
Resultado:
2
4
6
8
10
Error muy común: Muchos escriben:
```
for(int i = 1; i <= 5; i++)
```
porque piensan: el vector tiene 5 elementos
Pero eso genera:
vector[1]
vector[2]
vector[3]
vector[4]
vector[5]
Y:
vector[5]
NO existe.
Recordá:
Vector de tamaño 5
Índices válidos:
0
1
2
3
4
Analogía cotidiana
Imaginemos una fila de casilleros:
0 1 2 3 4
Recorrer el vector significa caminar frente a cada casillero uno por uno.
Voy al 0
Voy al 1
Voy al 2
Voy al 3

Voy al 4

Y en cada uno hago algo:

- leer un valor
- mostrarlo
- modificarlo
- compararlo

---

4. BUSCAR ELEMENTOS EN UN VECTOR

Definición

Buscar en un vector significa recorrer sus elementos para encontrar uno que cumpla una determinada condición.

Por ejemplo:

- Buscar el mayor número.
- Buscar el menor número.
- Buscar el primer número par.
- Buscar cuántos números son positivos.
- Buscar si existe un valor determinado.

¿Por qué necesitamos recorrer el vector?

Supongamos:

int numeros[5] = {12, 8, 25, 3, 17};

Si te pregunto: ¿Cuál es el número menor?

La computadora no lo sabe automáticamente.

Debe revisar:

12

8

25

3

17

uno por uno.

Por eso siempre que buscamos algo en un vector aparece un recorrido.

Buscar el menor elemento, supongamos:

int numeros[5] = {12, 8, 25, 3, 17};

Queremos obtener: 3

Estrategia

La computadora no puede comparar todos los números al mismo tiempo.

Entonces hacemos:

1. Supongo que el primero es el menor.
2. Recorro el resto del vector.
3. Si encuentro uno más chico, actualizo el menor.
4. Al finalizar, tendré el menor real.

Implementación

```
int menor = numeros[0];
for(int i = 1; i < 5; i++)
{
  if(numeros[i] < menor)
  {
     menor = numeros[i];
  }
}
```

Simulación paso a paso

Vector: 12 8 25 3 17

Inicialmente:

menor = numeros[0];

Entonces:

menor = 12

Iteración 1

i = 1

Comparamos:

8 < 12 ?

Sí.

Entonces:

menor = 8

Iteración 2

i = 2

Comparamos:

25 < 8 ?

No.

No cambia nada.

Iteración 3

i = 3

Comparamos:

3 < 8 ?

Sí.

Entonces:

menor = 3

Iteración 4

i = 4

Comparamos:

17 < 3 ?

No.

No cambia nada.

Resultado final: menor = 3

Buscar el mayor elemento, es exactamente la misma idea, pero modificando el signo < por > para que ahora busque el mayor

Buscar el primer número par, supongamos:

int numeros[5] = {7, 11, 8, 15, 20};

Queremos encontrar 8 porque es el primer número par.

```
for(int i = 0; i < 5; i++)
{
  if(numeros[i] % 2 == 0)
  {
     printf("%d", numeros[i]);
     break;
  }
}
```

¿Qué es break?

break significa:

Salir inmediatamente del ciclo.

Entonces:

7 → no es par

11 → no es par

8 → es par

Lo mostramos y terminamos.

Contar elementos

A veces no queremos encontrar un número.

Queremos contar cuántos cumplen una condición.

Ejemplo:

¿Cuántos números son pares?

```
int contador = 0;
for(int i = 0; i < 5; i++)
{
  if(numeros[i] % 2 == 0)
  {
     contador++;
  }
}
```

Vector:

12 7 8 15 20

Pares:

- 12

- 8
- 20

Resultado: contador = **3**

Patrón general de búsqueda

Casi todos los ejercicios de búsqueda siguen esta estructura:

```
for(int i = 0; i < tamaño; i++)
{
  if(condicion)
  {
     hacer_algo;
  }
}
```

Ejemplos:

if(numeros[i] < menor)

if(numeros[i] > mayor)

if(numeros[i] % 2 == 0)

if(numeros[i] == valor_buscado)

Relación con el modelo de examen

En el examen aparece algo parecido a: “Encontrar el menor número par del vector.”

Para resolverlo hay que combinar:

Recorrer

for(...)

Verificar si es par

numero % 2 == 0

Buscar el menor

if(numero < menor)

Es decir, el ejercicio mezcla varios conceptos que ya conocemos.

---

5. ORDENAR UN VECTOR

Definición

Ordenar un vector significa reorganizar sus elementos siguiendo algún criterio.

Los criterios más comunes son:

- De menor a mayor.
- De mayor a menor.

Ejemplo

Vector original:

8 3 10 1 5

Ordenado de menor a mayor:

1 3 5 8 10

Ordenado de mayor a menor:

10 8 5 3 1

¿Cómo ordenamos?

La computadora no sabe automáticamente cuál es el menor o el mayor.

Debe comparar los elementos entre sí.

Podemos pensarlo como ordenar cartas en la mano.

Tomamos una carta y la comparamos con las demás.

Si encontramos una que debería ir antes, las intercambiamos de lugar.

Intercambiar dos valores

Supongamos:

int a = 10;

int b = 20;

Queremos que queden:

a = 20;

b = 10;

Primer intento **(incorrecto)**

a = b;

b = a;

Veamos qué pasa:

a = 10

b = 20

Primera línea:

a = b;

Ahora:

a = 20

b = 20

Segunda línea:

b = a;

Ahora:

a = 20

b = 20

Perdimos el valor 10.

Solución: variable auxiliar

Necesitamos una variable temporal.

int aux;

aux = a;

a = b;

b = aux;

Paso a paso:

Inicialmente:

a = 10

b = 20
Guardamos temporalmente:
aux = a; → (aux = 10)
a = 10
b = 20
Luego:
a = b;
aux = 10
a = 20
b = 20
Finalmente:
b = aux;
aux = 10
a = 20
b = 10
Intercambio realizado correctamente.
Aplicando esto a un vector
Supongamos:
int numeros[5] = {8, 3, 10, 1, 5};
Queremos ordenarlo de menor a mayor.
¿Por qué aparecen dos for?
Porque cada elemento debe compararse con todos los demás.
Por eso usamos:

```
for(...)
{
  for(...)
  {
     ...
  }
}
```

A esto se lo llama: **For anidado** o **For dentro de otro for**
Ordenamiento de menor a mayor

```
int aux;
for(int i = 0; i < 5; i++)
{
  for(int j = i + 1; j < 5; j++) → la iteración del 2do for estará una posición delante del otro
  {
     if(numeros[j] < numeros[i]) → recordemos que numeros es el nombre del vector
     {
        aux = numeros[i];
```

```
            numeros[i] = numeros[j];
            numeros[j] = aux;
        }
    }
}
```

Entendiendo qué hacen i y j

i: Representa la posición que estamos intentando acomodar.

j: Recorre las posiciones que están después de i.

Por ejemplo:

8 3 10 1 5

Primera vuelta:

i = 0 → Estamos trabajando con: 8

Ahora: j = 1

Comparación:

3 < 8 ? Sí. Entonces intercambiamos.

El vector queda:

3 8 10 1 5

Luego:

j = 2

Comparación:

10 < 3 ? No.

Luego:

j = 3

Comparación:

1 < 3 ? Sí. Intercambiamos.

Vector:

1 8 10 3 5

Y así continúa.

Resultado final

Vector inicial:

8 3 10 1 5

Vector ordenado:

1 3 5 8 10

Ordenar de mayor a menor

Simplemente cambiamos el operador.

Antes:

`if(numeros[j] < numeros[i])`

Ahora:

`if(numeros[j] > numeros[i])`

Patrón importante

Cada vez que veas:

```
for(...)
{
  for(...)
  {
    if(...)
    {
      intercambio
    }
  }
}
```

muy probablemente estés viendo un algoritmo de ordenamiento.

Relación con el modelo de examen

En el modelo aparece: “Ordenar el vector de menor a mayor.”

Para resolverlo hay que saber:

1. Recorrer vectores.
2. Utilizar for anidados.
3. Comparar elementos.
4. Intercambiar usando una variable auxiliar.

---

6. ¿QUÉ ES UNA MATRIZ?

Definición

Una matriz es una estructura que permite almacenar muchos datos del mismo tipo organizados en filas y columnas.

Podemos pensar una matriz como una tabla de Excel.

Por ejemplo:

Dimension: 3x3	Columna 0	Columna 1	Columna 2
Fila 0	10	20	30
Fila 1	40	50	60
Fila 2	70	80	90

Tiene:

- 3 filas
- 3 columnas

Por eso la dimensión es 3x3

Relación con los vectores

Un vector almacena datos en una sola dimensión:

int numeros[5];
Visualmente:
[10][20][30][40][50]
Una matriz almacena datos en dos dimensiones:
int matriz[3][3];
Visualmente:
[10][20][30]
[40][50][60]
[70][80][90]
¿Cuándo usar una matriz?
Cuando necesitamos representar información organizada en filas y columnas.
Ejemplos:

- Notas de alumnos.
- Tableros de juegos.
- Asientos de un cine.
- Horarios escolares.
- Planillas de datos.

Declaración de una matriz
Sintaxis:
tipo nombre[filas][columnas];
Ejemplo:
int matriz[3][4];
Significa:
3 filas
4 columnas
Total:
3 × 4 = 12 posiciones
Inicialización
Podemos cargar valores al crearla.

```
int matriz[2][3] =
{
  {1, 2, 3},
  {4, 5, 6}
};
```

Visualmente:
1 2 3
4 5 6
Índices de una matriz
Al igual que los vectores:
Las posiciones comienzan en 0.

Ejemplo:

```
int matriz[2][3] =
{
  {1, 2, 3},
  {4, 5, 6}
};
```

Visualmente:

	0	1	2
0	**1**	**2**	**3**
1	**4**	**5**	**6**

Acceder a una posición

Sintaxis:

matriz[fila][columna]

Ejemplos:

matriz[0][0] → elemento en la fila 0 y columna 0

Resultado: **1**

matriz[0][2] → elemento en la fila 0 y columna 2

Resultado: **3**

matriz[1][1] → elemento en la fila 1 y columna 1

Resultado: **5**

Modificar elementos

Supongamos: matriz[1][1] = 100; → el valor de la fla 1 y columna 1 ahora es **100**

La matriz queda:

```
1   2    3
4 100   6
```

Recorrer una matriz

Acá aparece algo MUY importante.

Para recorrer una matriz necesitamos: **Dos for**

Porque:

- Un for recorre **filas**.
- Otro for recorre **columnas**.

Mostrar todos los elementos

```
for(int fila = 0; fila < 2; fila++)
{
  for(int columna = 0; columna < 3; columna++)
  {
     printf("%d ", matriz[fila][columna]);
  }
  printf("\n");
}
```

¿Qué hace cada for?

Primer for:

for(int fila = 0; fila < 2; fila++)

Recorre:

- fila 0
- fila 1

Segundo for:

for(int columna = 0; columna < 3; columna++)

Recorre:

- columna 0
- columna 1
- columna 2

Simulación

Primera vuelta:

fila = 0

Segundo for:

columna = 0

columna = 1

columna = 2

Muestra:

1 2 3

Luego:

fila = 1

Muestra:

4 5 6

Resultado final:

1 2 3

4 5 6

Cargar una matriz con scanf

```
int matriz[2][3];
for(int fila = 0; fila < 2; fila++)
{
  for(int columna = 0; columna < 3; columna++)
  {
     printf("Ingrese un numero: ");
     scanf("%d", &matriz[fila][columna]);
  }
}
```

Buscar un elemento en una matriz

Supongamos:

```
int mayor = matriz[0][0];
for(int fila = 0; fila < 2; fila++)
{
  for(int columna = 0; columna < 3; columna++)
  {
     if(matriz[fila][columna] > mayor)
     {
        mayor = matriz[fila][columna];
     }
  }
}
```

¿Qué hace?

Recorre toda la matriz.

Cada vez que encuentra un número más grande:

mayor = matriz[fila][columna]; → lo almacena en la variable **mayor**

Al finalizar: mayor contiene el valor más grande de toda la matriz.

Diferencia clave entre vector y matriz

Vector:

int numeros[5];

Se accede con:

numeros[i]

Matriz:

int matriz[3][4];

Se accede con:

matriz[fila][columna]

Analogía cotidiana

Un vector es como una fila de casilleros:

[ ] [ ] [ ] [ ] [ ]

Una matriz es como un edificio de casilleros:

[ ] [ ] [ ]

[ ] [ ] [ ]

[ ] [ ] [ ]

Ahora necesitamos dos coordenadas para encontrar algo:

Fila

Columna

---

7. 1. DIAGONAL PRINCIPAL

¿Qué es la diagonal principal?

La diagonal principal de una matriz está formada por todos los elementos cuya fila y columna tienen el mismo índice.

Es decir: **matriz[fila][columna]**

pertenece a la diagonal principal cuando: **fila == columna**

Ejemplo

Matriz:

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

Los elementos de la diagonal principal son: **1 5 9**

Porque están en:

matriz[0][0]

matriz[1][1]

matriz[2][2]

Observá que:

fila = columna

en todos los casos.

Recorrer la diagonal principal

Supongamos:

```
int matriz[3][3] =
{
   {1, 2, 3},
   {4, 5, 6},
   {7, 8, 9}
};
```

Podemos recorrerla con un único for:

```
for(int i = 0; i < 3; i++)
{
   printf("%d\n", matriz[i][i]); → printea solos que la i sea igual
}
```

Salida:

1

5

9

¿Por qué funciona?

Primera iteración:

i = 0

Accede a:

matriz[0][0]

Resultado:

1

Segunda iteración:

i = 1

Accede a:

matriz[1][1]

Resultado:

5

Tercera iteración:

i = 2

Accede a:

matriz[2][2]

Resultado:

9

Sumar la diagonal principal

Muchas veces los ejercicios piden calcular la suma.

Ejemplo:

```
int suma = 0;
for(int i = 0; i < 3; i++)
{
   suma += matriz[i][i];
}
printf("%d", suma);
```

Resultado:

1 + 5 + 9 = **15**

Patrón importante

Si ves: matriz[i][i] → estás trabajando con la diagonal principal.

7.2 DIAGONAL SECUNDARIA

¿Qué es la diagonal secundaria?

La diagonal secundaria es la otra diagonal de la matriz (que no es la principal) va desde el extremo derecho al izquierdo, de arriba hacia abajo

Ejemplo

Matriz:

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

La diagonal secundaria es: **3 5 7**

Porque corresponde a:

matriz[0][2]

matriz[1][1]

matriz[2][0]

Ahora **NO** coincide fila = columna.

¿Cómo identificarla?

En una matriz de tamaño: 3 x 3

los elementos de la diagonal secundaria cumplen:

fila + columna = 2

Comprobémoslo:

matriz[0][2]

0 + 2 = 2 ✔

matriz[1][1]

1 + 1 = 2 ✔

matriz[2][0]

2 + 0 = 2 ✔

Fórmula general

Para una matriz cuadrada de tamaño N (es decir si es 3x3, N = 3):

fila + columna = N - 1

o equivalentemente:

columna = N - 1 - fila

Recorrer la diagonal secundaria

Supongamos:

```
int matriz[3][3] =
{
  {1, 2, 3},
  {4, 5, 6},
  {7, 8, 9}
};
```

Podemos recorrerla con:

```
for(int i = 0; i < 3; i++)
{
  printf("%d\n", matriz[i][2 - i]);
}
```

Salida:

3

5

7

¿Por qué funciona?

Primera iteración

i = 0

Accede a: matriz[0][2]

Resultado: **3**

Segunda iteración

i = 1

Accede a: matriz[1][1]

Resultado: **5**

Tercera iteración

i = 2

Accede a: matriz[2][0]

Resultado: **7**

Forma general

Si la matriz es:

int matriz[N][N];

la diagonal secundaria se recorre así:

```
for(int i = 0; i < N; i++)
{
   printf("%d\n", matriz[i][N - 1 - i]);
}
```

Sumar la diagonal secundaria

```
int suma = 0;

for(int i = 0; i < 3; i++)
{
   suma += matriz[i][2 - i];
}
printf("%d", suma);
```

Resultado:

3 + 5 + 7 = **15**

Truco para acordarse

La principal forma una línea así: \

La secundaria forma una línea así: /

---

8. ESTRUCTURAS (STRUCT)

¿Qué es una estructura?

Definición

Una estructura (struct) es un tipo de dato que nos permite agrupar varias variables relacionadas entre sí bajo un mismo nombre.

¿Por qué existen?

Hasta ahora vimos variables como:

```
int edad;
float altura;
```

char inicial;

Cada variable almacena un único dato.

Pero imaginemos que queremos representar un alumno.

Un alumno tiene:

- Nombre
- Edad
- Promedio
- División

Si usamos variables sueltas:

int edad;

float promedio;

char division;

Nos falta saber que todas pertenecen al mismo alumno.

Ahí aparecen las estructuras.

Analogía cotidiana: Podemos pensar una estructura como una ficha de datos.

Por ejemplo:

Alumno

Nombre: Juan

Edad: 16

Promedio: 8.5

Division: A

Todos esos datos pertenecen al **mismo** alumno.

Sintaxis

```
struct Alumno
{
  int edad;
  float promedio;
  char division;
};
```

Analizando la estructura

struct: Indica que estamos creando una estructura.

Alumno: Es el nombre de la estructura.

edad: Campo de tipo int.

promedio: Campo de tipo float.

division: Campo de tipo char.

Importante: Esto **NO** crea un alumno.

Solamente define el **"molde"**.

Es parecido a cuando declaramos una función:

int doble(int numero). No ejecuta nada. Solo define algo.

Crear una variable de tipo struct

Ahora sí podemos crear alumnos.

struct Alumno alumno1;

Visualmente:

alumno1

- edad
- promedio
- division

Acceder a los campos

Se utiliza el operador punto (.)

alumno1.edad = 16;

alumno1.promedio = 8.5;

alumno1.division = 'A';

¿Qué significa el punto?

Se lee como:

alumno1.edad → La edad de alumno1

Alumno1.promedio → El promedio de alumno1

Mostrar datos

printf("%d\n", alumno1.edad);

printf("%.2f\n", alumno1.promedio);

printf("%c\n", alumno1.division);

Ejemplo completo

```
#include <stdio.h>
struct Alumno
{
  int edad;
  float promedio;
  char division;
};
int main()
{
  struct Alumno alumno1;
  alumno1.edad = 16;
  alumno1.promedio = 8.5;
  alumno1.division = 'A';
  printf("Edad: %d\n", alumno1.edad);
  printf("Promedio: %.2f\n", alumno1.promedio);
  printf("Division: %c\n", alumno1.division);
  return 0;
}
```

Varias estructuras

Podemos crear muchos alumnos.

struct Alumno alumno1;

struct Alumno alumno2;

struct Alumno alumno3;

Cada uno tendrá sus **propios datos.**

Vector de estructuras

Y acá aparece algo MUY importante para los exámenes.

Podemos tener:

struct Alumno alumnos[30];

¿Qué significa? 30 alumnos

Visualmente:

alumnos[0]

alumnos[1]

alumnos[2]

...

alumnos[29]

Cada alumno tiene:

- edad
- promedio
- division

Acceder a un alumno del vector

alumnos[0].edad → Se lee: La edad del alumno 0

alumnos[3].promedio → Se lee: El promedio del alumno 3

Cargar un vector de estructuras

```
for(int i = 0; i < 3; i++)
{
  printf("Edad: ");
  scanf("%d", &alumnos[i].edad);
  printf("Promedio: ");
  scanf("%f", &alumnos[i].promedio);
  printf("Division: ");
  scanf(" %c", &alumnos[i].division);
}
```

¿Cuándo usar struct?

Cuando varios datos pertenecen a una misma entidad.

Ejemplos:

Alumno:

```
struct Alumno
{
  int edad;
```

```
  float promedio;
};
```

Producto:

```
struct Producto
{
  int codigo;
  float precio;
};
```

Error común: edad = 16;

Pero edad pertenece a una estructura.

Debe ser: alumno1.edad = 16;

Porque necesitamos indicar de qué alumno estamos hablando.

---

9. MEMORIA DINÁMICA

¿Qué problema intenta resolver?

Hasta ahora, cuando creábamos un vector, teníamos que decidir su tamaño antes de ejecutar el programa.

Ejemplo:

```
int numeros[100];
```

Esto significa: Reservar espacio para 100 números enteros.

Pero muchas veces no sabemos cuántos datos vamos a necesitar.

Por ejemplo:

¿Cuántos alumnos hay?

El usuario podría responder: 5 o 50 o 500

Entonces surge un problema:

¿Cómo creamos un vector si no sabemos su tamaño?

Para resolver esto existe la **memoria dinámica.**

Memoria estática vs memoria dinámica

Memoria estática

```
int numeros[10];
```

Características:

- El tamaño se define al escribir el programa.
- No puede cambiar.
- La memoria se reserva automáticamente.

Memoria dinámica

```
int *numeros;
```

Características:

- El tamaño se decide mientras el programa se ejecuta.
- La memoria se reserva manualmente.
- Permite crear estructuras más flexibles.

Analogía cotidiana

Imaginemos que organizamos una fiesta.

Memoria estática:

Reservás: 10 sillas antes de saber cuántas personas vendrán.

Si vienen: 5 personas sobran sillas.

Si vienen: 20 personas faltan sillas.

Memoria dinámica:

Esperás a saber cuántas personas vienen.

Luego reservás exactamente: 20 sillas. No sobran ni faltan.

¿Dónde se guarda la memoria dinámica?

La memoria dinámica se reserva en una zona especial llamada: **Heap**

Por ahora no hace falta profundizar mucho.

Simplemente pensá: **Heap = lugar donde se guarda la memoria dinámica.**

¿Cómo se utiliza?

Para trabajar con memoria dinámica aparecen dos funciones muy importantes:

- **malloc()**
- **free()**
- **calloc()**
- **realloc()**

---

9. 1. MALLOC()

malloc()

Su nombre viene de: Memory Allocation, que significa: Asignación de memoria

¿Qué hace? Reserva memoria mientras el programa se está ejecutando.

Ejemplo:

```
int *vector;
vector = malloc(5 * sizeof(int));
```

Tranqui. Todavía no importa entender cada detalle.

La idea general es: Reservar espacio para 5 enteros.

¿Qué devuelve malloc? Devuelve la dirección de memoria donde se reservó el espacio.

Por eso necesitamos:

```
int *vector;
```

Más adelante veremos qué significa exactamente el asterisco.

Por ahora podemos pensar: vector apunta al bloque de memoria que reservó malloc.

¿Qué es sizeof()?

Permite saber cuántos bytes ocupa un tipo de dato.

Ejemplos:

```
sizeof(int)
sizeof(float)
sizeof(char)
```

No hace falta memorizar cuántos bytes ocupa cada tipo.
La idea es: sizeof(tipo) → ¿Cuánto espacio ocupa ese tipo de dato?
Ejemplo completo
Supongamos:
int cantidad;
printf("Cantidad: ");
scanf("%d", &cantidad);
Usuario: 8
Ahora podemos crear un vector exactamente del tamaño necesario:
int *vector;
vector = malloc(cantidad * sizeof(int));
Si cantidad vale: 8
entonces malloc reserva espacio para: 8 enteros
¿Qué pasa si no liberamos la memoria? La memoria queda ocupada innecesariamente.
Es como alquilar una cancha: Terminaste de jugar, pero seguís pagando.
Por eso existe: free()

9.2 FREE()
¿Qué es free()?
Definición
La función free() se utiliza para liberar la memoria que fue reservada previamente con malloc().
¿Por qué existe?
Cuando usamos:
malloc()
el programa reserva memoria en el Heap.
Esa memoria NO se libera automáticamente.
Por eso, cuando terminamos de usarla, debemos devolverla al sistema utilizando:
free()
Analogía cotidiana
Imaginemos que alquilás una cancha de fútbol.
malloc() → Reservar la cancha
La usás durante un rato.
Cuando terminás:
free() → Devolver la cancha
Si no la devolvés:
La cancha sigue ocupada innecesariamente.
Con la memoria ocurre exactamente lo mismo.
Sintaxis
free(variable);

Ejemplo:

```
free(vector);
```

Ejemplo completo

```
int *vector;
vector = malloc(5 * sizeof(int));
free(vector);
```

¿Qué hace?

1. Reserva espacio para 5 enteros.
2. Libera ese espacio cuando deja de ser necesario.

¿Cuándo usar free()?

Regla simple:

Si usaste malloc() → deberías usar free()

Normalmente al final del programa.

Ejemplo práctico

```
int cantidad;
printf("Cantidad: ");
scanf("%d", &cantidad);
int *vector;
vector = malloc(cantidad * sizeof(int));
for(int i = 0; i < cantidad; i++)
{
   scanf("%d", &vector[i]);
}
free(vector);
```

Error común

Reservar memoria:

```
vector = malloc(10 * sizeof(int));
```

y olvidarse de: free(vector);

Otro error común

Liberar memoria dos veces.

```
free(vector);
free(vector);
```

Esto puede producir errores porque la memoria ya fue liberada.

Por ahora la regla es:

Cada malloc() debe tener un único free().

Relación entre malloc y free

Siempre pensalos juntos:

malloc() → Reserva memoria

free() → Libera memoria

9.3 CALLOC()

¿Qué es calloc()?

Definición

La función **calloc()** permite reservar memoria dinámica para una cantidad determinada de elementos.

Se parece mucho a malloc(), pero tiene una diferencia muy importante.

Además de reservar memoria, **inicializa todos los valores en 0**.

¿Qué significa calloc?

Su nombre viene de: **Contiguous Allocation**

que significa: **Asignación continua de memoria.**

Sintaxis

calloc(cantidad_elementos, tamaño_de_cada_elemento);

Ejemplo:

```
int *vector;

vector = calloc(5, sizeof(int));
```

¿Qué hace?

Reserva memoria para:

- 5 enteros.

Y además:

Todos los elementos comienzan con valor **0**.

Podemos pensar que el vector queda así:

0 0 0 0 0

Diferencia entre malloc y calloc

Si hacemos:

```
int *vector;

vector = malloc(5 * sizeof(int));
```

Los valores iniciales son **desconocidos**.

Podrían ser:

154 -20 987 0 53

No debemos confiar en ellos.

Si hacemos:

```
int *vector;
```

```
vector = calloc(5, sizeof(int));
```

El vector comienza como:
0 0 0 0 0

Ejemplo completo

```
int *vector;

vector = calloc(5, sizeof(int));

for(int i = 0; i < 5; i++)
{
    printf("%d\n", vector[i]);
}

free(vector);
```

Salida:
0
0
0
0
0

¿Cuándo conviene usar calloc()?
Cuando queremos que la memoria comience con todos sus valores en **0**.
Así evitamos trabajar con valores basura.
Relación con malloc()
Ambas funciones sirven para reservar memoria.
La diferencia es:
malloc() → Reserva memoria.
calloc() → Reserva memoria e inicializa todos los elementos en 0.

9.4 REALLOC()
¿Qué es realloc()?
Definición
La función **realloc()** permite cambiar el tamaño de un bloque de memoria que fue reservado previamente con malloc() o calloc().
En palabras simples:
Sirve para **agrandar o achicar un vector dinámico**.

¿Qué significa realloc?
Su nombre viene de: **Reallocation**
que significa: **Reasignación de memoria.**
¿Por qué existe?
Supongamos que reservamos memoria para:
5 enteros

Pero luego necesitamos almacenar:
10 enteros

No hace falta crear otro vector desde cero.
Podemos cambiar el tamaño usando: realloc()

Sintaxis
realloc(variable, nuevo_tamaño);

Ejemplo:
vector = realloc(vector, 10 * sizeof(int));

¿Qué hace?
Si el vector tenía espacio para: 5 elementos
Ahora tendrá espacio para: 10 elementos

Ejemplo
int *vector;
vector = malloc(5 * sizeof(int));
vector = realloc(vector, 10 * sizeof(int));
free(vector);

¿Se pierden los datos?
No.
Los elementos que ya existían se conservan siempre que sea posible.
Por ejemplo:
Antes:
10
20
30
40
50

Después del realloc:
10
20
30
40
50
?
?
?
?
?

Los nuevos elementos **no están inicializados**.
También puede achicar memoria
Si tenemos espacio para: 10 elementos
Podemos reducirlo a: 5 elementos
Ejemplo:
vector = realloc(vector, 5 * sizeof(int));

Ejemplo completo
int *vector;
vector = malloc(3 * sizeof(int));
vector[0] = 10;
vector[1] = 20;
vector[2] = 30;
vector = realloc(vector, 6 * sizeof(int));
free(vector);

¿Cuándo usar realloc()?
Cuando durante la ejecución del programa descubrimos que necesitamos:

- Más memoria.
- Menos memoria.

Sin crear un vector completamente nuevo.
Relación entre malloc, calloc, realloc y free
Podemos pensarlas de esta forma:
malloc() → Reserva memoria.
calloc() → Reserva memoria e inicializa todo en 0.
realloc() → Cambia el tamaño de una memoria ya reservada.
free() → Libera la memoria reservada.

Resumen general:

Función	¿Qué hace?
malloc()	Reserva memoria.
calloc()	Reserva memoria e inicializa todos los elementos en 0.
realloc()	Cambia el tamaño de un bloque de memoria ya reservado.
free()	Libera la memoria reservada.

---

10. VECTORES DINÁMICOS

¿Qué es un vector dinámico?

Definición

Un vector dinámico es un vector cuyo tamaño se decide mientras el programa está ejecutándose.

A diferencia de los vectores estáticos, no necesitamos saber el tamaño cuando escribimos el programa.

Recordemos los vectores estáticos

Hasta ahora hacíamos:

int numeros[5];

Esto significa:

Crear un vector de 5 posiciones. El problema es que el tamaño queda fijo.

Si el usuario quiere guardar:

20 números → el vector anterior no alcanza.

Y si el usuario quiere guardar: 2 números → sobran posiciones.

El problema

Supongamos que queremos guardar las notas de los alumnos.

Primero preguntamos: ¿Cuántos alumnos hay? El usuario podría responder: 5 o 50 o 500

¿Cómo hacemos para crear un vector del tamaño exacto?

La solución: malloc()

Primero pedimos la cantidad:

```
int cantidad;
printf("Cantidad de alumnos: ");
scanf("%d", &cantidad);
```

Luego reservamos memoria:

```
int *notas;
notas = malloc(cantidad * sizeof(int));
```

¿Qué significa? Si cantidad vale: 5, se crea un vector para: 5 enteros

Si cantidad vale: 50, se crea un vector para: 50 enteros

Acceder a las posiciones

Se accede **exactamente igual** que en un vector normal.

Ejemplo:

```
notas[0] = 8;
notas[1] = 10;
notas[2] = 6;
```

También podemos mostrar valores:

```
printf("%d", notas[0]);
```

O recorrerlo:

```
for(int i = 0; i < cantidad; i++)
{
    printf("%d\n", notas[i]);
}
```

Cargar un vector dinámico

```
for(int i = 0; i < cantidad; i++)
{
    printf("Ingrese una nota: ");
    scanf("%d", &notas[i]);
}
```

Mostrar un vector dinámico

```
for(int i = 0; i < cantidad; i++)
{
    printf("%d\n", notas[i]);
}
```

Ejemplo completo

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int cantidad;
    printf("Cantidad de alumnos: ");
    scanf("%d", &cantidad);
    int *notas;
    notas = malloc(cantidad * sizeof(int));
    for(int i = 0; i < cantidad; i++)
    {
        printf("Nota %d: ", i + 1);
        scanf("%d", &notas[i]);
    }
```

```
    printf("\nNotas cargadas:\n");
    for(int i = 0; i < cantidad; i++)
    {
        printf("%d\n", notas[i]);
    }
    free(notas);

    return 0;
}
```

Nueva biblioteca (la que ustedes ya conocen)

Para usar malloc() y free() debemos agregar:

#include <stdlib.h>

Esta biblioteca contiene las funciones relacionadas con la memoria dinámica.

Comparación rápida

Vector estático

int numeros[10];

Ventajas:

- Más simple.
- Fácil de usar.

Desventajas:

- Tamaño fijo.

Vector dinámico

int *numeros;

numeros = malloc(cantidad * sizeof(int));

Ventajas:

- Tamaño elegido por el usuario.
- Usa solamente la memoria necesaria.

Desventajas:

- Requiere malloc().
- Requiere free().
- Es un poco más complejo.

Error común

Muchos creen que porque usamos memoria dinámica se accede distinto. **No.**

Después de hacer: notas = malloc(cantidad * sizeof(int));

seguimos usando:

notas[0]

notas[1]

notas[2]

exactamente igual que en los vectores estáticos.
